

Project Report

Traffic Correlation-Based Location Estimation in Anonymous Networks

Terminal-First and Full-Stack Cybersecurity Research Platform

Project Domain:	Cybersecurity / Network Traffic Analysis
Project Type:	Research Prototype and Full-Stack Demonstration System
Frontend Stack:	Next.js, Tailwind CSS
Backend Stack:	Express.js, WebSockets, SQLite
Analysis Engine:	Python 3

Submitted By:	_____
Roll Number:	_____
Department:	_____
Institution:	_____
Guided By:	_____
Academic Year:	_____

March 26, 2026

Title Page Note

This document presents the design, implementation, evaluation, and usage of a cybersecurity research platform titled *Traffic Correlation-Based Location Estimation in Anonymous Networks*. The report has been prepared in a form suitable for academic evaluation, project submission, and viva presentation. The implementation described in this report is aligned with a controlled and ethical research setting. It does not decrypt traffic, does not claim exact identification of real users, and does not represent unauthorized surveillance of anonymous networks. Instead, it demonstrates how metadata analysis, timing correlation, packet-size similarity, flow grouping, and path reconstruction can be used to estimate probable origin regions under simulated, replay, PCAP, and live-capture conditions.

Certificate

This is to certify that the report entitled *Traffic Correlation-Based Location Estimation in Anonymous Networks* is a bona fide record of the work carried out as part of a project in the domain of cybersecurity research and network traffic analysis. The work includes problem formulation, controlled-environment implementation, terminal-first analytics, a web-based dashboard, replay and PCAP processing, live traffic capture support, probabilistic correlation logic, and evaluation.

Project Guide Signature

Head of Department Signature

Declaration

I hereby declare that this report and the corresponding implementation are original work carried out for academic and research purposes. The references to published papers, documentation, and standard tools have been appropriately acknowledged. Any resemblance to practical operational surveillance systems should not be inferred; the system is designed and evaluated as a controlled cybersecurity analysis platform in accordance with ethical and defensive-security principles.

Acknowledgement

I would like to express sincere gratitude to my faculty members, project guide, and department for providing the opportunity to work on this project. The project demanded an interdisciplinary understanding of cybersecurity, networking, software engineering, and system design. I would also like to acknowledge the developers and maintainers of open-source tools such as Wireshark/TShark, Python, Node.js, Next.js, and Express.js, which made rapid experimentation and controlled research implementation possible.

Contents

List of Tables

List of Figures

Abstract

Anonymous communication systems are designed to conceal the origin of user traffic through layered routing and relay-based forwarding. While encryption protects the content of communication, metadata such as packet timing, packet length, protocol behavior, and burst structure can still reveal useful signals for defensive traffic analysis. This project presents a controlled cybersecurity platform that estimates probable origin regions from anonymized or anonymization-style network traffic using traffic correlation techniques. The system does not attempt to decrypt payloads or identify real users; instead, it evaluates probabilistic relationships between entry-side, middle-hop, and exit-side traffic features under simulation, replay, PCAP, and live-capture workflows.

The implementation is divided into three major layers. The first layer is a Python analysis engine responsible for feature extraction, hop-level similarity scoring, path construction, and probabilistic region estimation. The second layer is an Express.js backend that exposes REST APIs, upload endpoints, authentication, SQLite-backed persistence, and WebSocket-based session updates. The third layer is a Next.js and Tailwind CSS dashboard that allows the user to run simulations, upload replay files, process PCAP traces, and inspect ranked correlated paths and estimated regions. The system supports deterministic replay datasets as well as real packet metadata ingestion using TShark.

The report explains the motivation for the project, the architecture, mathematical formulation, implementation details, testing strategy, ethical boundaries, results, limitations, and future scope. It further demonstrates that a terminal-first core can be progressively expanded into a full-stack academic project while preserving the explainability of the analysis pipeline. The final outcome is a defensible and presentation-ready research platform that showcases how correlation-based inference can be explored in a controlled environment.

Chapter 1

Introduction

1.1 Background

Anonymous networking systems such as Tor are designed to provide communication privacy by separating the user's original identity from the final communication endpoint. These systems typically route traffic through multiple intermediate nodes, reducing the ability of any single observer to directly connect a sender with a receiver. However, privacy systems do not entirely eliminate metadata. Packet timing, packet sizes, sequence behavior, burst intervals, and routing structure may still carry signals that can be measured and correlated. This creates an important research problem in the field of cybersecurity: how to estimate traffic origin probabilistically using metadata while respecting ethical and technical limitations.

1.2 Problem Statement

The central problem addressed by this project is the following: given observed network traffic that has passed through an anonymous-routing-style system, can a controlled research platform estimate a probable source region using only traffic metadata and correlation methods? The project does not attempt exact identification, payload inspection, or decryption. Instead, it studies whether entry-side and downstream flow patterns can be matched through a scoring framework that measures timing similarity, packet-size similarity, and sequence continuity.

1.3 Project Objective

The objective of the project is to build an end-to-end analysis platform that:

- processes simulated, replayed, PCAP, and live-capture traffic metadata,
- extracts structured packet and flow features,
- correlates candidate hop transitions using transparent formulas,
- constructs likely routing paths,
- estimates probable origin regions with confidence scores,
- presents results in both terminal and browser-based interfaces.

1.4 Report Organization

This report begins with motivation and literature grounding, then explains the functional requirements and architecture derived from the PRD. It proceeds to discuss the implementation of the analysis engine, backend, and frontend. Later chapters describe testing, usage instructions, observed results, limitations, ethics, and future scope. The final chapter provides a concise conclusion and IEEE-style references.

Chapter 2

Motivation and Need for the Study

2.1 Motivation

The motivation for this project is rooted in the intersection of cybersecurity research and privacy-preserving network systems. Modern anonymous-routing systems are highly effective at protecting communication content, but cybersecurity analysts, researchers, and academic evaluators are still interested in how metadata may be studied under controlled conditions. This is especially relevant in areas such as threat intelligence, botnet behavior analysis, suspicious routing-pattern monitoring, and laboratory study of traffic-analysis resilience.

2.2 Academic Motivation

From an academic perspective, the project is valuable because it combines:

- network traffic fundamentals,
- statistical correlation methods,
- packet-capture tooling,
- backend system design,
- frontend dashboard development,
- ethical framing in cybersecurity research.

This makes it suitable as a final-year project or research-driven demonstration platform.

2.3 Practical Motivation

In practical terms, the project demonstrates how traffic metadata can be structured and analyzed without violating the scope of ethical defensive-security work. It provides a platform that can be shown to a teacher, evaluator, or project review panel not just as an idea, but as a working system with simulation, replay, PCAP upload, and live capture support.

2.4 Why a Full-Stack Implementation Was Important

Initially, a terminal-only implementation was sufficient for algorithm validation. However, to make the system presentation-ready and more useful for interactive inspection, a web-based layer became necessary. The full-stack version allows non-technical evaluators to observe session creation, path ranking, region confidence, and report outputs visually, which increases the clarity and educational value of the project.

Chapter 3

Product Requirements and Scope

3.1 PRD Alignment

The implemented system follows the PRD idea of a layered pipeline:

Packet Source → Feature Extraction → Correlation Engine → Location Estimation → Visualization

The final implementation supports four traffic input modes:

- **Simulation mode:** deterministic synthetic Tor-like routing data,
- **Replay mode:** JSONL and CSV event datasets,
- **PCAP mode:** packet capture files parsed using TShark,
- **Live mode:** timed packet capture from a selected interface.

3.2 In-Scope Features

- traffic metadata analysis,
- correlation-based routing-path inference,
- probabilistic origin-region estimation,
- terminal and browser-based output,
- persistent session storage,
- protected backend access,
- multi-user role-based access for admin, analyst, and viewer accounts,

- analysis of replay files and PCAPs,
- live capture support using host tools,
- interactive topology visualization using a dedicated graph library,
- exportable CSV and server-generated PDF reports.

3.3 Out-of-Scope Features

- packet payload decryption,
- direct user identification,
- real-world deanonymization claims,
- unauthorized surveillance,
- exploitation of anonymity networks.

Chapter 4

Literature Review

4.1 Traffic Analysis in Anonymous Networks

Traffic analysis has long been studied as a side-channel problem in anonymous communication systems. Even when content is encrypted, timing and volume patterns can leak information. Research has shown that low-latency networks are particularly susceptible to timing-based attacks if a sufficiently capable observer is able to compare ingress and egress flow properties. However, practical accuracy depends on network conditions, congestion, path churn, and measurement noise.

4.2 Correlation Methods

Classic traffic-correlation methods compare streams by:

- packet inter-arrival times,
- packet lengths or burst volumes,
- ordered sequence relationships,
- timing-window overlap,
- statistical distance measures between flows.

This project adopts a transparent weighted-similarity approach because it is explainable, reproducible, and suitable for an academic demo.

4.3 Operational Limitation in Literature

Published research repeatedly highlights the gap between controlled experiments and real-world deployment. In real networks, packet reordering, buffering, NAT behavior, multiplexing, protocol diversity, and measurement asymmetry make exact attribution extremely difficult. This report therefore uses the literature not to overstate capability, but to justify why probabilistic estimation and controlled experimentation are the appropriate project targets.

Chapter 5

System Architecture

5.1 High-Level Architecture

The project is implemented in three major layers:

1. Python analysis engine
2. Express backend services
3. Next.js frontend dashboard

5.2 Python Analysis Engine

The analysis engine includes the following modules:

- `capture/simulate.py`
- `capture/replay.py`
- `capture/pcap.py`
- `capture/live.py`
- `processing/features.py`
- `processing/correlator.py`
- `processing/graph.py`
- `processing/estimator.py`
- `processing/evaluator.py`

5.3 Backend Layer

The backend is implemented in Node.js using Express.js. It includes:

- login endpoint,
- admin-only user management endpoints,
- session creation endpoints,
- replay and PCAP upload endpoints,
- live-capture initiation endpoint,
- CSV and PDF export endpoints,
- SQLite-backed persistent storage,
- background job queue,
- WebSocket updates for session state.

5.4 Frontend Layer

The frontend is implemented using Next.js and Tailwind CSS. It includes:

- login page,
- simulation panel,
- replay/PCAP upload panel,
- live-capture controls,
- session list,
- role-aware controls and admin user-management panel,
- React Flow-based topology graph,
- ranked path tables,
- confidence chart and score plot,
- CSV/PDF export controls,
- terminal report panel.

Chapter 6

Mathematical Model and Core Algorithms

6.1 Feature Representation

Each event in the analysis engine is represented with the following key fields:

- timestamp,
- packet size,
- source and destination IP,
- node type,
- session identifier,
- sequence number,
- protocol,
- port metadata,
- flow identifier.

6.2 Time Similarity

The time similarity function is defined as:

$$\text{time_similarity} = \frac{1}{1 + \frac{\Delta t}{T}}$$

where Δt is the time difference between two candidate events and T is the configured time threshold.

6.3 Size Similarity

The packet-size similarity is defined as:

$$\text{size_similarity} = \frac{1}{1 + \frac{\Delta s}{S}}$$

where Δs is the absolute packet-size difference and S is the configured size threshold.

6.4 Sequence Similarity

The sequence relationship is modeled as:

$$\text{sequence_similarity} = \frac{1}{1 + \text{sequence_gap}}$$

This ensures that adjacent sequence relationships contribute higher scores than distant ones.

6.5 Final Correlation Score

The final score combines weighted similarities:

$$\text{score} = w_1 \cdot \text{time_similarity} + w_2 \cdot \text{size_similarity} + w_3 \cdot \text{sequence_similarity}$$

In implementation, the score is further adjusted by:

- session consistency bonus,
- label consistency bonus,
- capture-specific protocol bonus.

6.6 Path Construction

Candidate paths are built from:

- ENTRY $\rightarrow$ MIDDLE
- MIDDLE $\rightarrow$ EXIT

When complete three-hop evidence is unavailable, the graph stage may preserve partial paths to avoid discarding useful origin-side signals from sparse captures.

6.7 Region Estimation

The final region estimate is obtained by aggregating support from all ranked paths associated with candidate entry regions:

$$\text{confidence}(r) = \frac{\text{support}(r) \cdot \text{prior}(r)}{\sum_k \text{support}(k) \cdot \text{prior}(k)}$$

This produces a normalized probability-like ranking over candidate regions.

Chapter 7

Implementation Details

7.1 Simulation Mode

Simulation mode generates synthetic Tor-like sessions with entry, middle, and exit nodes. Suspicious sessions are injected deterministically at a fixed rate to make controlled testing possible. This provides reproducibility, which is useful for debugging and demonstration.

7.2 Replay Mode

Replay mode reads structured datasets from JSONL and CSV files. These datasets were created to verify that the analysis engine produces stable outputs and that the frontend/backend integration preserves consistency across repeated runs.

7.3 PCAP Mode

PCAP mode uses TShark to read packet capture files and export selected metadata fields. The parser extracts protocol, addresses, ports, timestamps, and frame lengths. The implementation then:

1. groups packets into flow keys,
2. segments them using idle gaps,
3. collapses them into burst-level structures,
4. converts one rich cluster into multiple candidate sessions,
5. maps those sessions into entry/middle/exit evidence for downstream correlation.

This part was refined during development because real PCAP files initially produced over-compressed outputs. In the final version, protocol fingerprinting was expanded to distinguish patterns such as TLS 1.3, TLS 1.2, HTTP/2, QUIC, DNS over HTTPS, and DNS over TLS wherever sufficient packet metadata is available.

7.4 Live Mode

Live mode captures packet metadata for a user-specified duration using TShark. The output is then processed through the same burst and clustering logic as PCAP mode. Live mode is useful for demonstration because traffic can be generated during the capture window, for example using `ping`, `curl`, or DNS queries.

7.5 Persistent Backend

The backend uses SQLite for persistence. It stores:

- users,
- auth tokens,
- audit logs,
- sessions,
- jobs.

This ensures that session history, authentication state, and administrative audit trails remain available even after multiple requests and page refreshes. The final implementation also enables SQLite WAL mode, foreign-key enforcement, and a busy timeout so the local database behaves more reliably under repeated dashboard activity. The schema was also prepared for future PostgreSQL migration by keeping the stored structures explicit and portable.

7.6 Authentication

A token-based authentication mechanism protects the API. The final implementation supports seeded administrative, analyst, and viewer accounts, with role-aware route protection, token expiry, explicit logout, and token revocation. The frontend performs login

and then sends the issued token automatically with authenticated requests. Administrative users can additionally create new accounts and update roles or passwords through the dashboard, while security-relevant actions are captured in the audit log.

7.7 Queue and Job Lifecycle

Each analysis request becomes a queued job. The job runner marks jobs as queued, running, completed, or failed, and the improved version also supports retry attempts and stale-job recovery so interrupted work does not remain permanently stuck in the running state. This is especially useful for live capture mode where analysis is not instantaneous.

Chapter 8

Frontend Design and Visualization

8.1 Design Goals

The frontend was designed with the following goals:

- simplicity for academic presentation,
- visual clarity,
- immediate visibility of session status,
- readable metrics and ranked paths,
- compatibility with both desktop and browser-based demos.

8.2 Dashboard Components

The dashboard includes:

- command center panel,
- session cards,
- metric summary cards,
- estimated region table,
- evaluation snapshot,
- confidence distribution chart,
- path score curve,

- React Flow-based interactive topology map,
- protocol fingerprint summary cards,
- ranked path table,
- hop-correlation table,
- CSV and PDF export panel,
- admin user-management panel,
- audit log review panel,
- terminal report panel.

8.3 Why Both Terminal and Web Output Are Kept

The terminal output remains important because it is compact, reproducible, and useful for debugging. The browser dashboard, however, improves accessibility and presentation quality. Together, they make the system suitable for both engineering validation and faculty demonstration.

Chapter 9

How the Project Was Built

9.1 Development Strategy

The project was built in stages:

1. terminal-first prototype,
2. modular Python analysis engine,
3. structured replay support,
4. full-stack integration,
5. persistent backend,
6. upload and live capture support,
7. iterative refinement of PCAP processing.

9.2 Key Engineering Decisions

- Explainability was prioritized over opaque machine learning.
- A weighted scoring system was chosen because it is easier to justify in a viva.
- SQLite was selected for simplicity and portability.
- TShark was used instead of writing a raw packet parser from scratch.
- The frontend was built using Next.js App Router for a modern single-project dashboard architecture.

9.3 Challenges Faced

- maintaining consistency between simulation, replay, PCAP, and live modes,
- avoiding over-correlation across unrelated sessions,
- interpreting sparse real captures,
- preventing stale frontend build issues,
- preserving ethical scope while discussing anonymous-network traffic analysis.

Chapter 10

How to Run the Project

10.1 Prerequisites

The following tools are required:

- Python 3,
- Node.js and npm,
- TShark for PCAP and live capture support,
- a modern browser.

10.2 Environment Files

Create backend configuration:

```
cp backend/.env.example backend/.env
```

Create frontend configuration:

```
cp frontend/.env.example frontend/.env.local
```

10.3 Install Dependencies

```
npm --prefix backend install  
npm --prefix frontend install
```

10.4 Run Backend

```
npm --prefix backend run dev
```

10.5 Run Frontend

```
npm --prefix frontend run dev
```

10.6 Open Dashboard

Open:

```
http://localhost:3000
```

10.7 Default Login

Default credentials for demo use:

```
username: admin  
password: admin123
```

10.8 CLI Test Commands

Simulation

```
python3 main.py --mode simulate --sessions 16 --top-k 6
```

Replay

```
python3 main.py --mode replay --dataset tests/sample_data/replay_dataset.jsonl  
python3 main.py --mode replay --dataset tests/sample_data/replay_dataset.csv
```

PCAP

```
python3 main.py --mode pcap --dataset tests/sample_data/better_capture2.pcap
```

Live

```
python3 main.py --mode live --interface any --capture-seconds 8
```

10.9 Testing Commands

```
python3 -m unittest discover -s tests -v  
npm run benchmark:python
```


Chapter 11

Experimental Results

11.1 Simulation Results

Simulation mode consistently produces meaningful correlations and stable region estimates because the dataset is deterministic and controlled. This mode is the strongest for demonstrating the mathematical correctness of the pipeline.

11.2 Replay Results

Replay mode verifies that file-based workflows behave consistently and produce reproducible outputs. JSONL and CSV versions of the same dataset were used to confirm parser consistency.

11.3 PCAP Results

PCAP mode was the most challenging part of the project. Early versions over-compressed captures and produced too few events. After parser refinement, real captures such as `better_capture2.pcap` began producing:

- non-zero raw event counts,
- hop correlations,
- ranked complete paths,
- non-empty region estimates.

This makes PCAP analysis suitable for demonstration, although its interpretive quality remains more variable than replay or simulation.

11.4 Live Results

Live mode works best when the user deliberately generates traffic during the capture window. Longer capture windows without traffic may lead to sparse outputs. For reliable demos, short live captures accompanied by `ping` and `curl` activity are recommended.

11.5 Observed Strengths

- modular and explainable pipeline,
- full-stack interface,
- support for multiple analysis modes,
- usable session persistence,
- real integration with TShark,
- interactive topology visualization,
- protocol-aware capture fingerprinting,
- role-aware multi-user access,
- exportable analyst-facing reports.

11.6 Observed Weaknesses

- PCAP/live outputs remain sensitive to traffic sparsity,
- evaluation metrics are more reliable on labeled datasets,
- probabilistic estimates should not be over-interpreted as ground truth.

Chapter 12

Testing and Validation

12.1 Unit and Integration Testing

Automated tests were written for:

- simulation pipeline,
- replay pipeline,
- CSV and JSONL consistency,
- PCAP parser clustering behavior,
- multi-seed simulation benchmark regression.

12.2 Benchmarking

To strengthen the evaluation story beyond single-case screenshots, the project now includes a repeatable Python benchmark command that runs the simulation pipeline across multiple random seeds and reports aggregate accuracy and top-region hit rate. This does not turn the project into a formal research benchmark suite, but it does provide a more defensible regression baseline for future tuning.

12.3 Manual Validation

Manual validation included:

- CLI runs,

- backend endpoint verification,
- login and session creation checks,
- upload testing in the browser,
- live capture demonstrations,
- repeated refinement of real PCAP files.

12.4 Validation Philosophy

The validation strategy used in this project emphasizes:

- reproducibility on synthetic and replay data,
- functionality on PCAP and live inputs,
- explicit acknowledgement of heuristic limits in capture-based inference,
- conservative interpretation of outputs,
- alignment with defensive-security research.

Chapter 13

Ethics, Security, and Limitations

13.1 Ethical Boundary

This system is intended for controlled cybersecurity analysis, teaching, and laboratory demonstrations. It is not a tool for payload decryption, direct identity exposure, or real-world deanonymization.

13.2 Security Considerations

The backend uses authentication, token expiry, logout-based token revocation, role-based access control, local persistence, audit logging, and server-side job handling. Uploaded files are processed locally. However, production hardening such as secure secret management, stronger password policy enforcement, and hardened deployment has not been the primary focus of this academic implementation.

13.3 Limitations

- estimates are probabilistic, not definitive,
- real-world traffic remains noisy and incomplete,
- PCAP and live outputs depend strongly on capture quality,
- current evaluation metrics are strongest on labeled replay/simulation data,
- region inference should be interpreted as indicative rather than exact.

Chapter 14

Future Scope

14.1 Short-Term Improvements

- richer PCAP feature extraction,
- deeper protocol-specific flow modeling beyond the current fingerprint layer,
- improved scoring for DNS and mixed-flow captures,
- richer analyst report templates,
- more advanced graph interaction and path filtering.

14.2 Medium-Term Improvements

- stronger password policy enforcement and account recovery flows,
- background worker scaling,
- full PostgreSQL migration for higher concurrency,
- authenticated report downloads.

14.3 Long-Term Research Scope

- machine-learning-based correlation scoring,
- formal benchmarking on larger labeled datasets,
- uncertainty visualization,

- adversarial robustness studies,
- distributed-monitoring simulation in laboratory environments.

Chapter 15

Conclusion

This project demonstrates that a traffic-correlation idea from a PRD can be transformed into a full working system with both analytical rigor and practical presentation value. The final platform includes simulation, replay, PCAP upload, and live capture analysis; an explainable scoring pipeline; protocol-aware capture fingerprinting; role-based multi-user access; exportable analyst reports; and a browser dashboard with an interactive topology graph suitable for faculty demonstration. The results confirm that metadata-based correlation can support probabilistic origin estimation in controlled environments, while also highlighting the practical limitations of extending such methods to real-world anonymous-network traffic. The project therefore succeeds as a cybersecurity research platform, an educational full-stack system, and a technically defensible academic submission.

Appendix A: Important Files

File	Purpose
main.py	CLI entry point
service.py	Shared analysis pipeline service
engine_api.py	JSON-producing engine wrapper
capture/simulate.py	Synthetic traffic generation
capture/replay.py	JSONL/CSV replay ingestion
capture/pcap.py	PCAP parsing and burst/session construction
capture/live.py	Live packet metadata capture
processing/correlator.py	Hop similarity scoring
processing/graph.py	Path construction
processing/estimator.py	Region estimation
backend/auth.js	Role-aware authentication and authorization middleware
backend/server.js	Backend API server
backend/db.js	SQLite persistence, token handling, and audit-log storage
backend/queue.js	Job queue logic
backend/pythonRunner.js	Python engine bridge
docker-compose.yml	Container orchestration for backend and frontend
backend/postgres_schema.sql	PostgreSQL-ready schema reference
frontend/app/page.js	Thin page entry point for the refactored dashboard
frontend/components/dashboard/index.js	Dashboard UI components
frontend/hooks/useDashboardClient.js	Client-side dashboard state and action hook
frontend/app/globals.css	Dashboard animation and global style layer
tests/test_benchmark_regression.py	Millionep simulation benchmark regression test
tests/benchmark_pipeline.py	Repeatable benchmark summary generator

Appendix B: Sample API Requests

```
curl http://127.0.0.1:4000/api/health
```

```
curl -X POST http://127.0.0.1:4000/api/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"admin123"}'
```

```
curl -X POST http://127.0.0.1:4000/api/sessions/simulate \  
-H "Authorization: Bearer <TOKEN>" \  
-H "Content-Type: application/json" \  
-d '{"sessions":8,"seed":7,"topK":4}'
```

References

Bibliography

- [1] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The Second-Generation Onion Router,” in *Proceedings of the 13th USENIX Security Symposium*, San Diego, CA, USA, 2004, pp. 303–320.
- [2] S. J. Murdoch and G. Danezis, “Low-Cost Traffic Analysis of Tor,” in *Proceedings of the 2005 IEEE Symposium on Security and Privacy*, Oakland, CA, USA, 2005, pp. 183–195.
- [3] K. Bauer, D. McCoy, D. Grunwald, T. Kohno, and D. Sicker, “Low-Resource Routing Attacks Against Tor,” in *Proceedings of the 2007 ACM Workshop on Privacy in the Electronic Society*, Alexandria, VA, USA, 2007, pp. 11–20.
- [4] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, “Website Fingerprinting in Onion Routing Based Anonymization Networks,” in *Proceedings of the 10th ACM Workshop on Privacy in the Electronic Society*, Chicago, IL, USA, 2011, pp. 103–114.
- [5] M. Perry, E. Clark, and S. Murdoch, “The Design and Implementation of the Tor Browser,” *Open Source Report*, The Tor Project, 2013.
- [6] T. Wang and I. Goldberg, “On Realistically Attacking Tor with Website Fingerprinting,” *Proceedings on Privacy Enhancing Technologies*, vol. 2016, no. 4, pp. 21–36, 2016.
- [7] W. Stallings, *Network Security Essentials: Applications and Standards*, 6th ed. Boston, MA, USA: Pearson, 2017.
- [8] The Tor Project, “Tor Metrics Portal,” [Online]. Available: <https://metrics.torproject.org/>. Accessed: 24-Mar-2026.
- [9] Wireshark Foundation, “Wireshark and TShark Documentation,” [Online]. Available: <https://www.wireshark.org/docs/>. Accessed: 24-Mar-2026.
- [10] P. Biondi, “Scapy,” [Online]. Available: <https://scapy.net/>. Accessed: 24-Mar-2026.

-
- [11] OpenJS Foundation, “Express.js Documentation,” [Online]. Available: <https://expressjs.com/>. Accessed: 24-Mar-2026.
 - [12] Vercel, “Next.js Documentation,” [Online]. Available: <https://nextjs.org/docs>. Accessed: 24-Mar-2026.
 - [13] Tailwind Labs, “Tailwind CSS Documentation,” [Online]. Available: <https://tailwindcss.com/docs>. Accessed: 24-Mar-2026.